ROADMAP

LangChain: Staged Learning Roadmap

An 8-stage, project-led path from your first tool-calling agent to a tested and deployable system.

Focus	Python · LangChain · LangGraph · LangSmith
Method	Simple mental models + hands-on exercises + official links
Source	Official LangChain documentation index
Edition	Python-focused · generated October 2026

How to use this PDF

Follow the sections in order, but build continuously. Every blue underlined title opens the corresponding official documentation page.

> Based on the current documentation available when generated. LangChain evolves quickly; use the linked changelog and documentation indexes to check updates.

Contents

Your Route at a Glance 5

The rule for this roadmap . . . 5

Goal . . . 6

Recommended Capstone: Support Desk Copilot . . . 6

How to Use This Roadmap . . . 6

Stage 1 — Understand the Stack and Run a First Agent 6

Learn . . . 7

LangChain, LangGraph, and LangSmith . . . 7

Python setup . . . 7

Build . . . 7

Why This Matters . . . 7

Completion Checks . . . 7

Official Documentation . . . 8

Stage 2 — Learn LangChain's Core Building Blocks 8

Learn . . . 8

Models . . . 8

Messages . . . 8

Tools . . . 8

Structured output . . . 9

Build . . . 9

Why This Matters . . . 9

Completion Checks . . . 9

Official Documentation . . . 9

Stage 3 — Add Memory, Context, Middleware, and Streaming 10

Learn . . . 10

Short-term memory . . . 10

Context engineering . . . 10

Middleware . . . 10

Streaming . . . 10

Build . . . 10

Why This Matters . . . 11

Completion Checks . . . 11

Official Documentation . . . 11

Stage 4 — Build a Small Retrieval-Augmented Knowledge Base 11

Learn . . . 11

The retrieval pipeline . . . 11

Build . . . 12

Why This Matters . . . 12

Completion Checks . . . 12

Official Documentation . . . 12
Stage 5 — Model the Workflow Explicitly with LangGraph 13
Learn . . . 13
When LangGraph becomes useful . . . 13
The graph mental model . . . 13
Graph API and Functional API . . . 13
Build . . . 13
Why This Matters . . . 14
Completion Checks . . . 14
Official Documentation . . . 14
Stage 6 — Make the Graph Durable and Human-Controlled 15
Learn . . . 15
Persistence . . . 15
Interrupts and human-in-the-loop . . . 15
Fault tolerance . . . 15
Streaming . . . 15
Build . . . 15
Why This Matters . . . 16
Completion Checks . . . 16
Official Documentation . . . 16
Stage 7 — Trace, Test, and Evaluate with LangSmith 17
Learn . . . 17
Observability . . . 17
Three testing layers . . . 17
Offline and online evaluation . . . 17
Build . . . 17
Why This Matters . . . 18
Completion Checks . . . 18
Official Documentation . . . 18
Stage 8 — Deploy, Monitor, and Choose an Advanced Path 19
Part A — Package and Deploy the Agent . . . 19
Learn . . . 19
Build . . . 19
Completion Checks . . . 20
Official Documentation . . . 20
Part B — Add Production Monitoring . . . 20
Build . . . 20
Completion Checks . . . 21
Official Documentation . . . 21
Part C — Choose One Advanced Track . . . 21
Track 1: Context Engineering and Custom Middleware . . . 21
Track 2: Guardrails and Safer Actions . . . 21

Track 3: Long-Term Memory . . . 22
Track 4: MCP . . . 22
Track 5: Multi-Agent Systems . . . 22
Track 6: Deep Agents . . . 23
Advanced-Track Completion Checks . . . 23
Final Capstone Acceptance Checklist **23**
Functionality . . . 23
Testing and Evaluation . . . 24
Observability and Operations . . . 24
Understanding . . . 24
Official Documentation Starting Points **24**

Your Route at a Glance

Stage	Main idea	Build result	Effort
1	Stack + first agent	One agent, one tool	6–8 h
2	Models, messages, tools, schemas	Typed support agent	8–10 h
3	Memory, context, middleware, streaming	Multi-turn agent	8–10 h
4	Retrieval and RAG	Grounded knowledge assistant	8–12 h
5	LangGraph workflow design	Explicit stateful graph	10–12 h
6	Durability, interrupts, retries	Safe resumable workflow	10–12 h
7	LangSmith tracing and evaluation	Measured quality pipeline	10–14 h
8	Deployment and monitoring	Production-ready service	12–16 h

Total: roughly 72–94 focused hours. At 6–8 hours per week, plan for 10–14 weeks. The completion checks matter more than the calendar.

The rule for this roadmap

Use one evolving Support Desk Copilot project. Spend about 30% reading and 70% building. Do not add multi-agent architecture, MCP, or Deep Agents until the single-agent baseline works and you can measure it.

Goal

Learn to build, test, observe, and deploy a production-style Python agent using the current LangChain stack:

- LangChain for models, tools, agent loops, memory, and middleware.
- LangGraph for explicit, stateful workflows that can pause, resume, retry, and recover.
- LangSmith for traces, tests, evaluations, production monitoring, and deployment.

This roadmap follows a 30% reading / 70% building split. Use one evolving project rather than starting a new tutorial at every stage.

Recommended Capstone: Support Desk Copilot

Build a support assistant that can:

1. Classify a support request.
2. Search a small product knowledge base.
3. Draft a structured answer.
4. Remember the current conversation.
5. Route requests through an explicit workflow.
6. Pause before sensitive actions.
7. Recover from temporary failures.
8. Stream progress to the caller.
9. Produce LangSmith traces and evaluation results.
10. Run locally as an Agent Server and, optionally, deploy to LangSmith Cloud.

Use fake or local tools for actions such as creating tickets or sending replies until the final stages.

How to Use This Roadmap

For each stage:

1. Read only the listed documentation.
2. Add the stage's feature to the same capstone.
3. Save evidence: sample runs, tests, traces, or evaluation results.
4. Complete every check before continuing.

Stay with one model provider at first. Do not introduce multi-agent architecture, MCP, or Deep Agents until the basic single-agent system is working and measurable.

Stage 1 — Understand the Stack and Run a First Agent

Suggested effort: 6–8 focused hours
Outcome: A small Python agent that can call one tool.

Learn

LangChain, LangGraph, and LangSmith

Use this simple mental model:

- LangChain is the high-level framework. It supplies standard interfaces for models, tools, agents, memory, and middleware.
- LangGraph is the lower-level runtime. Use it when you need explicit workflow steps, durable execution, persistence, or human approval.
- LangSmith is the development and production platform. It provides tracing, evaluation, monitoring, and deployment.
- Deep Agents is a higher-level harness. It adds planning, filesystem tools, subagents, and context management for more autonomous tasks. Leave it for the advanced stage.

LangChain agents already run on LangGraph, so you do not need to learn graph construction before creating your first LangChain agent.

Python setup

The current Python quickstart uses Python 3.11 or newer. Create an isolated environment, install `langchain`, and then install the package required by your chosen model provider.

Keep API keys in environment variables or a local `.env` file. Do not place secrets directly in Python source code or commit them.

Build

Create the first version of the Support Desk Copilot:

1. Define a simple `lookup_order_status(order_id: str)` function.
2. Turn it into a tool.
3. Create an agent with:
   - one model;
   - the order-status tool;
   - a short system prompt.
4. Invoke the agent with:
   - a question that does not need the tool;
   - a question that should call the tool;
   - an invalid order ID.
5. Print the final response and inspect the returned messages.

Keep the tool deterministic. A Python dictionary is enough for the first version.

Why This Matters

A LangChain agent is fundamentally a model running in a loop: the model decides whether to answer, call a tool, inspect the result, or continue. Understanding that loop prevents the framework from feeling like unexplained magic.

Completion Checks

- □ The project runs in a clean Python environment.
- □ No API key appears in source control.
- □ The agent answers a simple question without calling a tool.

- The agent calls `lookup_order_status` when order data is needed.
- Invalid tool input produces a controlled result rather than crashing.
- You can explain the difference between LangChain, LangGraph, LangSmith, and Deep Agents in one sentence each.

Official Documentation

- Runtimes, frameworks, and harnesses
- Install LangChain
- LangChain quickstart
- LangChain overview
- Python provider integrations

Stage 2 — Learn LangChain's Core Building Blocks

Suggested effort: 8–10 focused hours
Outcome: An agent with typed tools and predictable application output.

Learn

Models

The model is the agent's reasoning engine. LangChain provides a common interface across model providers, making it easier to change providers or compare models later.

Learn to:

- initialize a model;
- invoke it directly without an agent;
- configure basic model parameters;
- distinguish a standalone model call from an agent loop.

Messages

Messages carry the conversation and tool interactions. Learn the roles of:

- system messages;
- human messages;
- AI messages;
- tool messages.

Inspect message objects instead of treating every result as a plain string.

Tools

A tool is a typed function the model may choose to call. Its name, docstring, arguments, and return value all affect agent behavior.

Good beginner tools should:

- do one job;
- use type hints;

- have a clear, concise docstring;
- validate input;
- return useful error information;
- avoid hidden side effects.

Structured output

Natural-language answers are useful for people but awkward for application code. Structured output lets the agent return validated data such as a Pydantic model, dataclass, or JSON-compatible object.

Build

Expand the Support Desk Copilot:

1. Add tools such as:

- `lookup_order_status`;
- `search_product_policy`;
- `calculate_refund_window`.

2. Create a Pydantic output model such as:

``python class SupportDecision(BaseModel): category: Literal["order", "refund", "product", "other"] urgency: Literal["low", "medium", "high"] answer: str needs_human: bool ``

3. Configure the agent to return this structure.
4. Add input-validation and tool-error examples.
5. Log which tool was selected for each sample request.
6. Add a simple command-line interface that accepts a question and prints the validated result.

Why This Matters

Typed tools reduce ambiguity at the boundary between the model and your Python code. Structured output gives the rest of your application a stable contract instead of forcing it to parse prose.

Completion Checks

- □ You can call the model directly and through an agent.
- □ Every custom tool has type hints and a useful docstring.
- □ The model selects the correct tool for representative questions.
- □ Bad tool input is handled explicitly.
- □ The final result validates against a Pydantic model.
- □ Application code reads fields such as `category` and `needs_human` without parsing free text.
- □ You can identify system, human, AI, and tool messages in a completed run.

Official Documentation

- Models
- Messages
- Tools
- Agents
- Structured output

Stage 3 — Add Memory, Context, Middleware, and Streaming

Suggested effort: 8–10 focused hours
Outcome: A multi-turn agent whose behavior can be controlled around the agent loop.

Learn

Short-term memory

Short-term memory stores the state of one conversation thread. A checkpointer saves this state, while a `thread_id` identifies which conversation to load.

Do not confuse:

- Model context: what the model sees during one call.
- Agent state: data persisted during a conversation.
- Long-term memory: data retained across different conversation threads.

Context engineering

Reliable agents need the right information at the right time. Context engineering controls:

- instructions;
- message history;
- available tools;
- response format;
- runtime information;
- stored user information.

More context is not automatically better. Unrelated history can increase latency, cost, and mistakes.

Middleware

Middleware hooks into the agent lifecycle. It can add logging, change prompts, select models or tools dynamically, summarize history, enforce guardrails, and request human review.

Start with built-in middleware before writing custom middleware.

Streaming

Streaming lets the caller see tokens, tool activity, or progress before the whole run finishes. It improves perceived responsiveness and makes long-running work easier to understand.

Build

Add conversational behavior to the capstone:

1. Configure an in-memory checkpointer.
2. Pass a distinct `thread_id` for each conversation.
3. Demonstrate that:

- the same thread remembers earlier information;
- a new thread does not inherit that information.

4. Add runtime context such as:
- `user_id`;
- customer tier;
- preferred response style.
5. Use middleware to alter the system prompt based on that context.
6. Add streaming so the command-line client shows progress or model output as it arrives.
7. If the conversation becomes long, experiment with message trimming or summarization middleware.

Why This Matters

Most real assistants are multi-turn systems. State and context must be managed deliberately; otherwise, conversations become expensive, irrelevant history accumulates, and users' data can leak between sessions.

Completion Checks

- □ The same `thread_id` preserves conversation context.
- □ A different `thread_id` starts an independent conversation.
- □ Runtime context changes agent behavior without being hardcoded into the tool.
- □ At least one middleware component is active.
- □ The caller receives visible streamed progress or output.
- □ You can explain the difference between state, runtime context, and long-term storage.
- □ Tests or manual checks prove that one user's thread does not reuse another user's history.

Official Documentation

- Short-term memory
- Context overview
- Context engineering in agents
- Middleware overview
- Built-in middleware
- Streaming

Stage 4 — Build a Small Retrieval-Augmented Knowledge Base

Suggested effort: 8–12 focused hours
Outcome: Answers grounded in retrieved product documentation.

Learn

The retrieval pipeline

Learn these components in order:

1. Documents: text plus metadata such as source and page.
2. Document loaders: read source files.
3. Text splitters: divide large documents into useful chunks.

4. Embeddings: convert text into vectors.
5. Vector stores: save vectors and find similar text.
6. Retrievers: provide a standard search interface.
7. RAG: pass retrieved material to a model or agent to answer a question.

Retrieval does not guarantee correctness. It supplies potentially relevant evidence. Your application still needs useful chunking, metadata, prompts, tests, and traces.

Build

Add a product knowledge base:

1. Select a few small product or policy documents.
2. Load them as LangChain `Document` objects.
3. Preserve source metadata.
4. Split the documents into chunks.
5. Generate embeddings and index the chunks in a vector store.
6. Test semantic search independently from the agent.
7. Wrap search as a retriever or tool.
8. Require the Support Desk Copilot to:

- search when documentation is needed;
- answer from retrieved material;
- return source metadata;
- admit when the retrieved content does not support an answer.

Start with a small, inspectable collection. Do not add a large production database yet.

Why This Matters

A model’s built-in knowledge may be stale or unrelated to your private information. Retrieval gives the model selected, current application data without placing every document into every prompt.

Completion Checks

- □ Documents retain source metadata after loading and splitting.
- □ A direct similarity search returns sensible chunks for several queries.
- □ The agent uses retrieval for knowledge questions.
- □ The final answer includes the sources used.
- □ An unsupported question produces an honest fallback.
- □ You have at least one test where retrieval returns a distracting chunk and the agent avoids relying on it.
- □ You can inspect retrieved text separately from the model’s final answer.

Official Documentation

- Build a semantic search engine with LangChain
- Document loader integrations
- Text splitter integrations
- Embedding integrations

- Vector store integrations
- Retriever integrations
- Custom RAG agent with LangGraph

Stage 5 — Model the Workflow Explicitly with LangGraph

Suggested effort: 10–12 focused hours
Outcome: A visible graph with typed state, separate nodes, and conditional routing.

Learn

When LangGraph becomes useful

Use LangGraph when a single agent loop no longer gives enough control—for example, when the application needs:

- deterministic steps mixed with agent decisions;
- branching and loops;
- inspectable intermediate state;
- independent retry behavior;
- persistence;
- pause-and-resume behavior;
- long-running execution.

The graph mental model

Learn four ideas:

- State: the shared data used by the workflow.
- Nodes: Python functions that read state and return updates.
- Edges: connections between nodes.
- Conditional edges: routing decisions that choose the next node.

Keep state as raw data. Build prompts inside nodes instead of storing preformatted prompts in state.

Graph API and Functional API

Learn what both APIs are, but implement the capstone with the Graph API first. It makes nodes, state, and routing visible, which is useful while learning.

Build

Rebuild the capstone as an explicit workflow:

1. Define typed state for:

- original request;
- classification;
- retrieved documents;
- draft answer;
- approval status;

- error information.

2. Create separate nodes for:

- classifying the request;
- searching documentation;
- handling account or order lookup;
- drafting a response;
- deciding whether human review is required;
- producing the final result.

3. Add conditional routing based on category and urgency.
4. Draw the expected graph before writing it.
5. Compare the drawn graph with the compiled graph.
6. Test nodes individually with small state dictionaries.
7. Keep external API operations in separate nodes so failures can be handled independently.

Why This Matters

An explicit graph turns hidden control flow into an inspectable design. It becomes easier to understand why a path was chosen, test one step, and apply retries only where they are safe.

Completion Checks

- □ State has an explicit Python schema.
- □ Each node has one clear responsibility.
- □ At least one conditional branch is exercised.
- □ Raw data, not formatted prompts, is stored in state.
- □ A single node can be tested without running the whole graph.
- □ External-service work is separated from model-only work.
- □ You can draw the actual graph from memory and explain every transition.

Official Documentation

- LangGraph overview
- LangGraph quickstart
- Thinking in LangGraph
- Workflows and agents
- Graph API overview
- Use the Graph API
- Choosing between Graph and Functional APIs

Stage 6 — Make the Graph Durable and Human-Controlled

Suggested effort: 10–12 focused hours
Outcome: A workflow that can pause, resume, stream, and recover from expected failures.

Learn

Persistence

LangGraph has two complementary persistence mechanisms:

- Checkpointers save a thread's graph state. They support conversation continuity, recovery, human review, and time travel.
- Stores save application-defined information across threads, such as user preferences.

An in-memory implementation is suitable for learning and tests. Production systems need durable storage.

Interrupts and human-in-the-loop

Interrupts pause a graph and expose information to an external reviewer. The graph resumes later using the same thread and saved state.

Use approval for consequential actions such as:

- sending a message;
- changing a customer record;
- issuing a refund;
- deleting data;
- executing write queries.

Fault tolerance

Handle different failures differently:

- retry temporary network or rate-limit failures;
- use timeouts for slow operations;
- route exhausted retries to a recovery path;
- ask a person for missing information;
- allow unexpected programming errors to remain visible.

Retries require care around side effects. A retried "send" or "charge" operation must not accidentally happen twice.

Streaming

Stream state, messages, or progress events so the caller can see which part of the graph is active.

Build

Harden the capstone:

1. Compile the graph with a checkpointer.
2. Require a stable `thread_id`.
3. Add an interrupt before a mock `send_reply` or `issue_refund` action.

4. Support:
- approve;
- edit;
- reject.
5. Add a simulated unreliable API node.
6. Configure retries, timeout behavior, and a recovery route for that node.
7. Stream progress events such as:
- classifying;
- searching;
- drafting;
- waiting for approval;
- completed.
8. Verify that a paused run resumes from saved state instead of starting over.
9. If practical, try time travel by replaying or forking from a saved checkpoint.

Why This Matters

Production agents do not finish every task in one uninterrupted request. They wait for people, hit rate limits, lose network connections, and may run for a long time. Durability lets them continue safely instead of repeating the entire task.

Completion Checks

- □ A run pauses before a sensitive action.
- □ The saved thread resumes using the same `thread_id`.
- □ Approval executes the mock action.
- □ Rejection prevents it.
- □ Editing changes the proposed action before execution.
- □ A temporary failure is retried according to policy.
- □ A failure after all retries follows a visible recovery path.
- □ The client receives progress while the graph runs.
- □ The workflow cannot silently repeat a side effect during a retry.

Official Documentation

- Persistence
- Checkpointers
- Stores
- Interrupts
- Human-in-the-loop
- Fault tolerance
- LangGraph streaming
- Use time travel

Stage 7 — Trace, Test, and Evaluate with LangSmith

Suggested effort: 10–14 focused hours
Outcome: A measurable agent with repeatable tests, traces, and an evaluation dataset.

Learn

Observability

A LangSmith trace records the complete work performed for one request. Nested runs show model calls, tools, retrievers, and workflow steps.

Use traces to answer:

- What input did the model receive?
- Which tool did it select?
- What did the tool return?
- Which route did the graph take?
- Where did the latency or cost come from?
- Why did the final answer fail?

Learn the difference between runs, traces, threads, and trajectories.

Three testing layers

1. Unit tests: Fast tests for deterministic code, individual nodes, tools, routing functions, and schemas. Use fake models and in-memory persistence where appropriate.
2. Integration tests: Run against real model and service APIs. These catch schema, credential, latency, and provider-integration problems but cost more and can be flaky.
3. Evaluations: Measure behavior over a dataset. Evaluators may use deterministic checks, trajectory matching, human feedback, or an LLM judge.

Offline and online evaluation

- Offline evaluation runs against curated examples before release. Use it to compare versions and catch regressions.
- Online evaluation scores sampled production traces. Use it to detect quality problems after deployment.

Failures found in traces should become dataset examples so they are tested in future experiments.

Build

Create a quality pipeline for the capstone:

1. Enable LangSmith tracing.
2. Run representative requests and inspect the full trace tree.
3. Add unit tests for:

- tool validation;
- structured output;
- graph routing;
- approval policy;
- retry and recovery logic.

4. Add a small real-model integration suite.
5. Create a LangSmith dataset covering:
   - routine questions;
   - missing documentation;
   - ambiguous requests;
   - incorrect order IDs;
   - urgent requests;
   - requests that need approval;
   - attempts to bypass approval.
6. Add deterministic evaluators for:
   - expected category;
   - required source fields;
   - whether human review was requested;
   - forbidden tool use.
7. Add one quality evaluator, such as an LLM judge, for answer usefulness or groundedness.
8. Run a baseline experiment.
9. Change one prompt, model, tool description, or retrieval setting.
10. Run a second experiment and compare quality, latency, and cost.
11. Turn at least one traced failure into a permanent regression example.

Why This Matters

Agent output is nondeterministic, and a change in one component can affect the entire execution path. Testing individual functions is necessary but not enough; traces and dataset-based evaluations show how the complete agent behaves.

Completion Checks

- □ Every capstone run produces a readable LangSmith trace.
- □ Model, tool, retriever, and graph steps appear in the trace.
- □ Deterministic logic has unit tests.
- □ Real external integrations have a separate integration-test path.
- □ A LangSmith dataset contains both normal and failure cases.
- □ At least one deterministic evaluator checks an exact requirement.
- □ At least one evaluator measures a qualitative requirement.
- □ Two experiments can be compared.
- □ A real failure has been added as a regression example.
- □ You can state which version is better and support the decision with evaluation results.

Official Documentation

- LangSmith Observability
- Tracing quickstart

- Observability concepts
- Testing LangChain agents
- Unit testing
- Integration testing
- Agent evaluations
- LangSmith Evaluation
- How to evaluate agents
- LangGraph testing

Stage 8 — Deploy, Monitor, and Choose an Advanced Path

Suggested effort: 12–16 focused hours
Outcome: A locally served or cloud-deployed agent with production monitoring and one carefully chosen advanced capability.

Part A — Package and Deploy the Agent

Learn

LangSmith Deployment runs agents through Agent Server. The deployment stack supports durable execution, streaming, persistence, and horizontal scaling.

Understand these pieces:

- application source and graph entry point;
- dependency configuration;
- Agent Server;
- LangGraph CLI;
- LangSmith Studio;
- local development versus production-like local testing;
- deployment control plane and data plane.

Use `pyproject.toml` or `requirements.txt` consistently. Keep runtime secrets outside the repository.

Build

1. Organize the capstone as a deployable application.
2. Declare its Python dependencies.
3. Configure the graph entry point.
4. Run it locally with the supported LangGraph development tooling.
5. Test:

- a normal run;
- streaming;
- conversation threads;
- an interrupted run;

- resuming that run.

6. If a LangSmith Cloud workspace is available, complete the cloud deployment quickstart.
7. Call the deployed agent through the SDK or API rather than only through Studio.

Completion Checks

- [ ] A clean environment can install the declared dependencies.
- [ ] The application starts through the documented local server workflow.
- [ ] The server accepts a request and returns a result.
- [ ] Streaming works through the server.
- [ ] Thread state persists as expected.
- [ ] An interrupted run can be resumed.
- [ ] No secret is embedded in source code.
- [ ] If using cloud deployment, a fresh deployed revision passes the same smoke tests.

Official Documentation

- LangSmith Deployment
- Deploy your app to cloud
- Agent Server overview
- Application structure
- Set up an application with `pyproject.toml`
- Set up an application with `requirements.txt`
- Local development and testing
- LangGraph deployment

Part B — Add Production Monitoring

Build

1. Open the deployed application's traces.
2. Use dashboards to watch:

- request volume;
- errors;
- latency;
- token usage;
- cost;
- tool performance;
- feedback scores.

3. Configure an online evaluator on a sample of production-style traces.
4. Create at least one alert, such as:

- error rate over a threshold;
- latency increase;

- cost spike;
- quality score decrease.

5. Verify the alert notification path with a test notification where supported.
6. Document how a failed production trace becomes an offline regression example.

Completion Checks

- ☐ Production-style requests appear in LangSmith.
- ☐ A dashboard shows latency, errors, usage, and quality signals.
- ☐ An online evaluator is attached to the correct traces.
- ☐ Sampling or filtering limits unnecessary evaluation cost.
- ☐ At least one alert is configured.
- ☐ There is a written trace-to-dataset feedback loop.

Official Documentation

- Monitor projects with dashboards
- Alerts in LangSmith
- Set up LLM-as-a-judge online evaluators
- Filter traces

Part C — Choose One Advanced Track

Do not implement every track immediately. Choose the one that solves a real limitation observed in the capstone's traces or evaluations.

Track 1: Context Engineering and Custom Middleware

Choose this when the agent has the correct capabilities but receives the wrong instructions, history, tools, or user information.

Build one of:

- dynamic prompts based on runtime context;
- dynamic tool selection;
- automatic conversation summarization;
- model selection based on task complexity;
- custom logging or validation middleware.

Docs:

- Context engineering in agents
- Custom middleware
- Prebuilt middleware

Track 2: Guardrails and Safer Actions

Choose this when the agent handles sensitive content or can take consequential actions.

Build a layered policy with:

- deterministic input checks;
- PII handling;

- output validation;
- human approval for sensitive tools;
- tests proving blocked operations do not run.

Docs:

- Guardrails
- Human-in-the-loop

Track 3: Long-Term Memory

Choose this when the assistant must remember information across separate conversation threads.

Store only information that has a clear purpose, such as user preferences. Keep thread history in the checkpointer and cross-thread data in a store.

Docs:

- Memory overview
- Long-term memory
- LangGraph stores

Track 4: MCP

Choose this when you need to connect the agent to tools supplied by one or more MCP servers instead of writing every integration directly.

Begin with one trusted server. Learn connection lifetime, tool discovery, authentication, and result handling before connecting many servers.

Docs:

- Model Context Protocol
- MCP tools
- MCP connections
- MCP authentication

Track 5: Multi-Agent Systems

Choose this only when traces show that one agent has too many tools, too much context, or genuinely separate specialist tasks.

Before adding agents, decide whether the real need is:

- a subagent;
- a handoff;
- an on-demand skill;
- a router;
- a custom LangGraph workflow.

A single agent with better tools or context is often simpler.

Docs:

- Multi-agent overview
- Subagents
- Handoffs

- Skills
- Router
- Custom workflow

Track 6: Deep Agents

Choose this for long, open-ended tasks that need planning, filesystem access, context management, or subagent delegation.

Do not migrate merely because the project is “advanced.” First confirm that the existing LangChain or LangGraph solution is becoming difficult to maintain.

Docs:

- Deep Agents overview
- Deep Agents quickstart
- Customize Deep Agents
- Deep Agents context engineering
- Deep Agents subagents
- Going to production

Advanced-Track Completion Checks

- □ The chosen feature solves a failure or limitation observed earlier.
- □ The simpler single-agent baseline remains available for comparison.
- □ New behavior has tests and evaluation examples.
- □ LangSmith traces clearly show the new execution path.
- □ Quality improved enough to justify any additional latency, tokens, cost, or complexity.
- □ You can explain why the other advanced tracks were not needed yet.

Final Capstone Acceptance Checklist

Functionality

- □ The system classifies support requests.
- □ It uses typed tools with validated inputs.
- □ It produces structured output.
- □ It searches a documented knowledge base.
- □ It returns source metadata.
- □ It maintains isolated conversation threads.
- □ It routes work through an explicit LangGraph workflow.
- □ It pauses before a sensitive action.
- □ It can resume a paused thread.
- □ It handles temporary failures without hiding unexpected ones.
- □ It streams useful progress.

Testing and Evaluation

- □ Deterministic tools and nodes have unit tests.
- □ Real provider integrations have separate integration tests.
- □ A LangSmith dataset covers normal, edge, and adversarial cases.
- □ Deterministic evaluators check exact requirements.
- □ A qualitative evaluator measures answer quality or groundedness.
- □ A baseline and improved experiment have been compared.
- □ At least one production-style failure became a regression test.

Observability and Operations

- □ Traces expose model, tool, retrieval, and graph activity.
- □ Errors, latency, tokens, cost, and feedback can be monitored.
- □ An online evaluator samples appropriate traffic.
- □ An alert covers one important operational or quality risk.
- □ Secrets are kept outside the source code.
- □ Dependencies and the graph entry point are declared.
- □ The application runs through Agent Server locally or in LangSmith Deployment.

Understanding

You are ready to design your own LangChain application when you can clearly explain:

- when a direct model call is enough;
- when to use `create_agent`;
- when an explicit LangGraph workflow is justified;
- the difference between state, checkpointers, stores, and runtime context;
- why a trace is not the same as an evaluation;
- the difference between offline and online evaluation;
- why multi-agent architecture is an option rather than a default.

Official Documentation Starting Points

These indexes are the source of truth for keeping the roadmap current:

- Complete LangChain documentation index
- Python build documentation index
- Python learning resources
- Python reference overview
- Python versioning policy
- Python changelog

Generated from the official LangChain documentation index and linked official pages. Links are intended for further exploration.